이지우 Lee Jiwoo

백엔드 개발자 · 세종대학교 지능기전공학부 2027.02 졸업예정

> **Spring으로 서비스를 만들고, 만난 문제는 재현해서 고칩니다.**

전시 추천 앱 **전록**과 외국인 도움 요청 플랫폼 **HelpMeKorea**를 스토어에 출시했습니다. 다중 인스턴스에서의 예약 작업 중복 실행과 개발 서버 로그의 동시성 오류는 재현해 원인을 확인한 뒤 고쳤고, 장애가 날 만한 지점은 설계 단계에서 실패 경로를 정리했습니다. 같은 방식으로 Spring 생태계 오픈소스에 **버그 수정 PR 7건**이 머지됐습니다.

Projects

프로젝트	소개	역할	핵심 사례
전록	전시 추천·기록 앱 (출시)	백엔드 단독 · 인프라	예약 작업 중복 실행(ShedLock), 웹훅 재처리, 추천 규칙
HelpMeKorea	외국인 도움 요청 플랫폼 (출시)	백엔드 (팀)	탈퇴 처리, 로그인 동시성, 요청 수락 경쟁, 평점 정합성
Cloumy	방한 외국인 AI 여행 앱	기획·설계·구현 (개인)	LLM 환각 검증, 프롬프트 주입, Redis 장애, 규칙 엔진
StackFlow	Spring 실행 흐름 분석 도구	단독	정적 분석 × OTel 대조
오픈소스	Spring Boot · Kafka · AI 등	기여	버그 수정 PR 7건 머지

Skills

Language	Java 21 · Python 3.11
Spring	Boot 3.x · Security(JWT · OAuth2 · JWKS) · Data JPA · QueryDSL · WebSocket(STOMP) · WebFlux(WebClient)
Data	MySQL · PostgreSQL(PostGIS · pgvector) · Redis · ShedLock
AI	FastAPI · LangChain · RAG(pgvector)
Infra	AWS(EB · ALB · ASG · S3 · CloudFront) · Docker · GitHub Actions · nginx
Quality	JUnit 5 · Testcontainers · k6 · pytest
Monitoring	OpenTelemetry · Micrometer · Prometheus

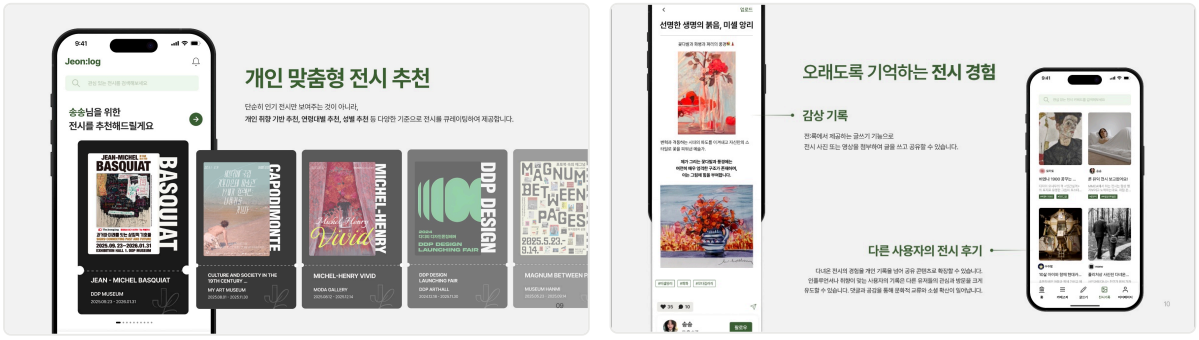
dlwldn30@naver.com | 010-2304-0895 | github.com/dlwldn30

전록 Jeonlog · 전시 추천·기록 서비스 App Store 출시·운영 중

2025.08 ~ 현재 · 팀 프로젝트(프론트 3 · 디자이너 2 · PM 1 · 백엔드 1) · 백엔드 단독 · 인프라 구성

App Store ↗ | GitHub Team-Jeonlog/Jeonlog-Backend ↗

기술 Java 21 · Spring Boot 3.3.5 · JPA · MySQL · Redis · ShedLock · k6 · AWS(EB · ALB · ASG)



추천 화면(좌) · 관람 기록과 후기 화면(우)

서버 여러 대에서 예약 작업 중복 실행 → 공유 DB 기반 ShedLock

문제 서버를 여러 대로 띄워 보니 인스턴스마다 같은 `@Scheduled` 작업을 독립적으로 실행하고 있었습니다. 전시 종료 알림은 `dedup_key` 유니크 제약 덕분에 한 번만 저장됐지만, 그 제약에 기대면 나머지 인스턴스는 매번 중복키 예외로 작업이 중단되고, 이런 제약이 없는 예약 작업은 같은 보호를 받지 못합니다.

해결 JVM 내부 락은 다른 인스턴스의 실행을 알 수 없어 제외하고, 이미 공유 중인 DB를 실행 권한의 결정 지점으로 삼아 ShedLock(JDBC)을 적용했습니다. `usingDbTime()`으로 서버 시계 차이를 피했고, 작업마다 락 이름과 유지 시간을 따로 두고 시간대는 Asia/Seoul로 맞췄습니다.

결과 **수정 전** 인스턴스 2대 → 같은 작업 2회 실행, 매 실행마다 `dedup_key` 중복 예외 1건 → **수정 후** 1회 실행, 예외 0건

알림 저장과 푸시 호출은 전후 모두 5건으로 같았습니다. `dedup_key` 제약이 중복 알림을 이미 막고 있었기 때문이며, 락의 효과는 중복 알림 차단이 아니라 중복 실행과 예외 롤백 제거입니다. 이후 웹훅 재처리와 지표 스냅샷에도 같은 락을 적용했습니다.

측정 같은 DB에 앱 2대를 동시에 띄워 10회차 반복(N=10) · 로컬 Docker MySQL, cron 2분 간격, FCM 스텁 · `75eac7a` → `ec7989e`

웹훅 실패가 로그에만 남음 → 실패 상태 저장·재처리

문제 신고·차단 알림을 보내는 Discord 웹훅 코드를 읽다 보니, 전송에 실패하면 로그만 남기고 끝나는 구조였습니다. 일시 오류와 재시도가 무의미한 오류도 구분하지 않아, 장애가 나면 알림이 그대로 유실될 수 있었습니다.

해결 실패를 버리지 않고 상태로 저장해 재처리하는 방식이 낫다고 판단했습니다. Resilience4j의 재시도는 상태를 메모리에 두어 재시작하면 사라지고 DB 상태 저장과 역할이 겹쳐, 메시지 큐도 쓰지 않고 DB 상태 저장만으로 구현했습니다. 재시도 불가 실패는 `FAILED`, 일시 실패는 `PENDING_RETRY`로 저장해 60초부터 backoff로 최대 5회 재전송하며, 재처리 스케줄러에는 ShedLock을 적용했습니다.

결과 **수정 전** 전송 실패 → 로그만 남고 유실 → **수정 후** `FAILED`·`PENDING_RETRY`로 저장되어 최대 5회 재전송

검증 단위 테스트 7개 — 재시도 불가 실패 저장, payload 길이 제한, 재처리 성공, 일시 오류 재예약

추천 집계 왜곡·결과 불안정·후보 부족 → 서비스 규칙으로 정리

문제 추천 코드에서 세 가지 문제가 보였습니다. 인기 집계에서 클릭 로그와 북마크를 함께 join해 행이 곱해지며 `COUNT`가 부풀 수 있었고, 정렬 기준 없는 limit과 시드 없는 무작위 보강으로 같은 사용자의 결과가 호출마다 달라졌습니다. 후보가 적으면 목표 개수를 채우지 못했습니다.

↳ 이어서 · 추천 집계 왜곡·결과 불안정·후보 부족 → 서비스 규칙으로 정리

해결 추천 점수보다 집계 정확성과 결과 재현성, 후보 부족 정책이 먼저라고 판단했습니다. 인기 집계는 `COUNT(DISTINCT …)`로 바꾸고 후보 쿼리에 날짜·ID 정렬을 추가했으며, 무작위 선택은 사용자 ID와 KST 날짜를 시드로 고정했습니다. 후보가 부족하면 진행 중 → 60일 내 예정 → 종료되지 않은 전시 → 전체 순으로 넓히고 중복을 제거해 목표 10개, 최소 7개를 채웁니다.

결과 **수정 전** 클릭 c건 × 북마크 b건을 join하면 행이 c×b개로 늘어 COUNT가 부풀음(클릭 3·북마크 2 → 둘 다 6, 점수 12), 랜덤 보강이 들어가는 사용자는 10회 호출에 10종류의 결과, 종료되지 않은 전시가 부족하면 목표 미달(진행 중 3개 + 종료 20개 → 3개 반환)

수정 후 `COUNT(DISTINCT)`로 클릭 3·북마크 2(점수 5)가 되어 인기 순위가 1위에서 2위로 내려가고, 사용자 ID·날짜 시드로 10회 호출 결과가 1종류, 종료 전시를 후순위로 채워 10개 반환

취향 데이터가 충분해 랜덤 보강이 없는 사용자는 수정 전에도 결과가 같았고, 불안정은 랜덤 보강 구간에서만 나타났습니다.

검증 단위 테스트 4개 — 후보 10개 반환, fallback 최소 개수 보강, 후보 부족, 단계 간 중복 제거. 최소 7개 보강은 단위 테스트로만 확인했습니다(실제 DB에서는 앞 단계 보강이 먼저 채움).

측정 로컬 Docker MySQL 8.0, 전시 70건 fixture, N=3 · `3a645e4` → `1549f07`

Cloumy 방한 외국인 대상 AI 여행 앱

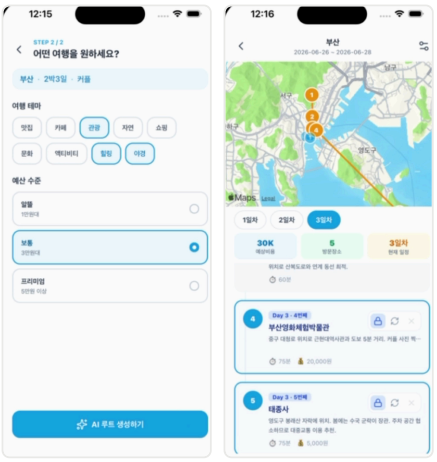
2026.04 ~ 현재 · 개인 프로젝트 · 기획 · 설계 · 구현 전체

GitHub Cloumy-app/cloumy ↗

기술 Spring Boot 3.3.5 · FastAPI · Claude API · OR-Tools · PostgreSQL 16(PostGIS · pgvector) · Redis · React Native

파이프라인 입력 → pgvector·PostGIS 후보 검색 → Day별 구역 클러스터링 → Claude(Sonnet)가 NDJSON 생성 → 슬롯 검증 → Day별 TSP(숙소 왕복)·이동시간 계산 → SSE 전송. 슬롯 대안·장소명 번역 같은 단순 작업은 Haiku로 분리했습니다.

입력 화면(좌)은 선택지만 서버로 보내고 프롬프트는 서버가 만듭니다. 생성 결과(우)의 장소는 DB 후보에서 고른 것입니다.



LLM 환각 장소 → 후보 선조회 + 생성 후 검증

문제 앱으로 일정을 생성해 테스트하던 중 Claude가 후보에 없는 `place_id`를 만들거나 같은 장소를 여러 번 배치하는 것을 확인했습니다. "후보만 사용하라"는 프롬프트를 강화해도 계속 재현됐습니다.

해결 프롬프트로는 막히지 않는다고 판단해 생성 결과를 서버에서 검증했습니다. 후보는 pgvector·PostGIS로 미리 뽑아 Day별 구역으로 나누고, 슬롯마다 `place_id`를 후보와 대조해 후보 밖이거나 중복인 장소는 그 구역의 미사용 후보로 교체하며, 교체할 후보가 없으면 슬롯을 제외합니다. 슬롯 수 상한도 프롬프트로는 지켜지지 않아(한 Day 6슬롯) 코드로 강제했고, 슬롯 대안은 모델이 후보 번호만 고르게 했습니다.

결과 **수정 전**(검증 전 모델 원본) 30회 생성 중 26회에서 후보 밖 `place_id`, 18회에서 같은 장소 중복, 19회에서 Day별 슬롯 상한(6개) 초과 → **수정 후**(검증 통과 후 최종 결과 30회 기준) 세 항목 모두 0회

동작 후보 밖·중복 장소는 교체하거나 제외하고, 후보가 없으면 모델을 호출하지 않음. 검증은 슬롯 단위, 전송은 Day 단위

모델이 같은 슬롯을 반복 출력해 위반 건수가 크게 부풀었기 때문에, 건수 대신 위반이 있었던 실행 수로 집계했습니다.

측정 `claude-sonnet-4-6` · 2026-10-12 · 부산 2박3일, density normal, 후보 80개 · 30회 모두 응답을 받음(2회는 max_tokens로 종료)

앱 안내 컨텍스트가 system prompt에 그대로 → type+params 계약

문제 앱을 돌려 테스트하다 보니 `proactive_context`가 자유 문자열 그대로 Spring과 FastAPI를 거쳐 system prompt 뒤에 붙고 있었습니다. 인증된 사용자가 서버 지시 위치에 임의의 문장을 넣을 수 있는 구조였습니다.

해결 금지 문구 필터는 우회될 수 있다고 판단해 클라이언트가 문장을 만들 수 없게 입력 계약을 바꿨습니다. `{type, params}`만 받아 서버가 문장을 조립하게 했으나 `reservationPlatform: str`로 악성 문장이 다시 섞였고, DB CHECK는 앱이 보낸 값을 검사하지 않아 방어가 되지 않았습니다. 그래서 Spring에서 type 화이트리스트와 `@Valid`로, FastAPI에서 type별 15종 discriminated union으로 검증하고, 값은 enum·UUID·시각 타입과 길이로 제한했으며 장소명 같은 자유 문자열은 서술에서 제외했습니다.

결과 **수정 전** 인증된 사용자가 보낸 임의 문장이 system prompt에 그대로 삽입 → **수정 후** 악성 문자열은 파싱 단계에서 거절되고, 서버가 조립한 문장만 프롬프트에 들어감

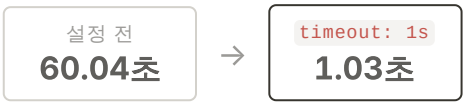
검증 type·params 불일치, unknown type, 형식 오류, 자유 문자열 제외를 테스트로 확인

Redis 장애 시 인증 응답 60.04초 → 1.03초

문제 출시 전 장애 시나리오 점검에서 Redis를 중단시켜 보니, 블랙리스트 조회가 예외 처리 없이 실행돼 정상 JWT 요청까지 필터 단계에서 예외로 끝났습니다. `@RestControllerAdvice`는 필터보다 뒤에서 동작해 이를 처리하지 못했고, 인증이 필요한 모든 요청이 500이 됐습니다.

해결 블랙리스트는 로그아웃한 토큰을 막는 보조 장치라, 이것 하나 때문에 전체 인증이 멈추는 것보다 위험이 작다고 판단해 조회 실패 시 통과시키는 fail-open으로 바꿨습니다. 이때 노출되는 범위는 Redis 장애 중에 로그아웃된 토큰이 만료되기 전까지로 한정됩니다. 다른 방법은 이번에는 적용하지 않았습니다. fail-closed는 토큰 하나를 막으려고 모든 인증 요청을 막는 셈이고, 로컬 폐기 캐시는 인스턴스별 메모리라 다른 인스턴스에 전파되지 않아 보호가 불완전하며, 짧은 토큰 TTL은 재발급 빈도를 높이고, Redis 분리·이중화는 지금 규모에서 운영 비용이 이득보다 크다고 봤습니다. 블랙리스트 조회만 분리해 폐기 토큰(`JWT_REVOKED`)은 `catch (Exception)`이 삼키지 않게 다시 던졌고, Redis 무응답 시 응답이 60.04초 걸려 Lettuce 기본 타임아웃을 1초로 줄였습니다.

결과



수정 전 Redis 장애 시 인증이 필요한 모든 요청이 500 → **수정 후** 정상 JWT는 통과

검증 폐기 토큰 거부와 장애 시 통과를 각각 단위 테스트로 확인

측정 `docker pause`로 Redis를 무응답으로 만든 로컬 측정 · N=5 중앙값 · `f96b947^` → `f96b947`

챗봇 선제 안내 → LLM 대신 규칙 엔진이 판단

문제 챗봇이 사용자 질문에만 응답해, 여행 전날의 날씨·일정 밀도나 출국 시각처럼 먼저 알려야 할 내용을 전달하지 못했습니다.

해결 신호가 날씨·시각·이동시간·예산뿐이라 LLM에 맡기면 뻔한 말에 그칠 수 있다고 판단해 규칙 엔진으로 개입 여부를 정했습니다. 날씨와 현재 슬롯 추정이 이미 FastAPI에 있어 Spring 대신 FastAPI에 두었고, 규칙 15종은 순수 함수로 서버는 `type`·`params`만 내려주며 문구는 앱 i18n이 만듭니다. 위치가 불확실하면 위치 의존 규칙은 건너뛰고 여행 기간 밖이면 쿼리 없이 종료하며, 후보를 모아 우선순위로 하나를 고르는 구조라 이 선택만 LLM으로 바꿀 수 있습니다.

결과 **수정 전** 사용자가 묻기 전에는 날씨·일정 밀도·출국 시각 안내가 없음 → **수정 후** 규칙 15종이 개입 여부를 판단해 하나를 선택, 이 판단에 LLM 호출이 없어 추가 지연·비용 없음

검증 규칙별 단위 테스트(`test_proactive_rules.py`)

HelpMeKorea 한국 거주 외국인 도움 요청 플랫폼 · 백엔드 스토어 출시

2026.07 ~ 현재 · 팀 프로젝트 · 백엔드 담당(프론트엔드와 협업) · 비공개(코드·설계 문서는 면접 시 공유 가능)

기술 Java 21 · Spring Boot 3.x · Security(JWKS) · JPA · PostgreSQL · Redis · STOMP · Testcontainers

탈퇴 후 계정 부활·통계 변동 → 익명화 혼합 삭제 + 재개 가능한 3단계

문제 탈퇴 처리를 설계하던 중 Supabase(신원)와 앱 DB(Postgres)를 분리하면서 FK CASCADE 정리가 사라진 점과, 앱 DB만 지우면 재로그인 때 같은 `sub`으로 유저가 다시 생성되는 문제를 확인했습니다.

해결 하드 삭제는 요청 기록이 사라져 헬퍼 통계가 낮아지고, 전면 soft delete는 조회마다 필터가 필요해 누락 시 탈퇴자가 노출돼 제외했습니다. 대신 남의 통계에 필요한 요청·게시글·댓글은 익명화하고 본인 데이터만 삭제하는 혼합 방식으로 정했습니다. 탈퇴 시작 마커 커밋 → 신원 삭제(이미 없으면 성공) → 앱 데이터 한 트랜잭션 정리 순으로 처리해, 신원을 먼저 지워 재로그인 부활을 막고 마커로 중간에 끊긴 계정을 찾아 재개할 수 있게 했습니다.

결과 **단순 구현** 앱 DB만 삭제하면 재로그인 때 같은 `sub`으로 계정 재생성, 하드 삭제하면 헬퍼 통계 하락 → **채택안** 재로그인 부활 차단, 상대방 이력·헬퍼 통계 유지, 중간에 끊겨도 재개

검증 통합 테스트(`AccountDeletionIT`) — 계정 부활 차단, 통계 불변, 부분 실패 후 재개

최초 로그인 동시 요청 500 → INSERT … ON CONFLICT로 해결

문제 개발 서버 로그에서 `users_pkey` 중복 키 오류를 확인했습니다. 최초 로그인 직후 `/me`·푸시 토큰 등록·알림 설정 조회가 모두 조회 후 INSERT하는 방식이라, 동시에 INSERT가 실행되면 한쪽이 500이 됐습니다.

해결 프론트 호출 순서 조정은 재시도와 다중 기기에서 재발하고, `synchronized`는 인스턴스 간 공유되지 않으며, 행 락은 공유되지만 아직 없는 행(최초 로그인)은 잠글 수 없습니다. 예외를 잡고 재조회하는 방식은 트랜잭션이 rollback-only가 될 위험이 있어, DB가 원자적으로 처리하는 `INSERT … ON CONFLICT`를 택했습니다. `users`는 `DO NOTHING` 후 조회해 탈퇴 마커가 있는 행이 되살아나지 않게 했고, 푸시 토큰만 마지막 등록자에게 소유권을 넘기도록 `DO UPDATE`를 썼습니다.

결과 **수정 전** 최초 로그인 직후 동시 요청이 겹치면 `users_pkey` 중복으로 한쪽이 500 — 20개 스레드 동시 생성 × 10회(200건) 재현에서 80~90건 실패(N=3) → **수정 후** 0건 실패(N=3), 행은 1개

측정 `UserService.provision`을 직접 동시 호출, Hikari 풀 3, 로컬 Docker Testcontainers PostgreSQL · `5214392` → `e118d03`

검증 Testcontainers PostgreSQL 통합 테스트(`UserProvisioningConcurrencyIT`) — 탈퇴 마커가 있는 행을 덮어쓰지 않는 것까지 확인

중복 요청·수락 경쟁 → 행 락 + 부분 유니크 인덱스로 막기

문제 요청 설계를 점검하던 중 같은 요청의 동시 수락이나 모바일 재시도로 활성 요청이 중복될 수 있음을 확인했습니다. 생성은 잠글 행이 없고 수락·완료·취소는 기존 요청의 상태 전이라 같은 방식으로 풀 수 없었습니다.

해결 조건부 `UPDATE`는 상태·참여자·활동 지역 검증을 따로 구성해야 해서, 락으로 직렬화하고 DB 제약으로 마무리하는 방식을 택했습니다. 생성은 사용자 행을 `FOR UPDATE`로 잠근 뒤 탈퇴 여부, 멱등성 키 재생(입력이 다르면 충돌), 활성 요청 존재를 차례로 확인하고, 멱등성 키 유일 인덱스와 `PENDING`·`MATCHED` 사용자당 1개 부분 유니크 인덱스를 두었습니다. 수락·완료·취소는 요청 행을 잠근 뒤 상태·참여자·활동 지역을 재검증하고, 기존 중복 데이터는 마이그레이션이 임의로 취소하지 않고 중단합니다.

결과 **단순 구현** 같은 요청의 동시 수락이나 모바일 재시도로 활성 요청이 중복될 수 있음 → **채택안** 행 락으로 직렬화하고, 멱등성 키 유일 인덱스와 `PENDING`·`MATCHED` 부분 유니크 인덱스를 DB가 최종 방어선으로 담당

검증 동시성 통합 테스트 6개 — 동시 수락, 같은 키·다른 키 동시 생성, 완료와 취소 경합, 수락과 헬퍼 비활성화 경합, 양쪽 동시 종료

평균 평점 오차 누적·근사 백필 → 합계 원천 + 새 마이그레이션으로 교정

문제 평점 집계를 점검하던 중 반올림된 평균으로 다음 평균을 계산하면 오차가 누적되고, `rating_sum` 추가 시 기존 평균에서 역산한 V9 백필이 이미 어긋난 값을 기준 데이터로 저장한다는 것을 확인했습니다. 4.78이 4.80으로 반올림된 헬퍼는 리뷰 20건일 때 합계가 96으로 고정돼 자격 없는 배지가 붙을 수 있었습니다.

해결 읽을 때마다 `AVG`를 계산하면 조회·배지 평가마다 집계 비용이 들어, 합계와 건수를 정수로 보존하고 평균은 합계÷건수로만 만들도록 했습니다. 적용 이력이 있을 수 있는 V9는 수정하지 않고 새 V12를 추가해, 완료·평점 있는 `help_requests`를 원본으로 헬퍼별·카테고리별 합계·건수·평균을 재계산했으며 단순 카운터는 건드리지 않았습니다.

결과 **수정 전** 반올림된 평균 4.80에서 역산해 합계가 96으로 고정(실제 평균 4.78, 리뷰 20건) → **수정 후** 완료·평점 있는 요청 원본에서 합계·건수·평균을 재계산

검증 평균 계산 단위 테스트, 재집계 SQL 통합 테스트

StackFlow Spring 실행 흐름 분석 도구

2026.07 ~ 현재 · 단독 · `docker compose up` 5분 데모

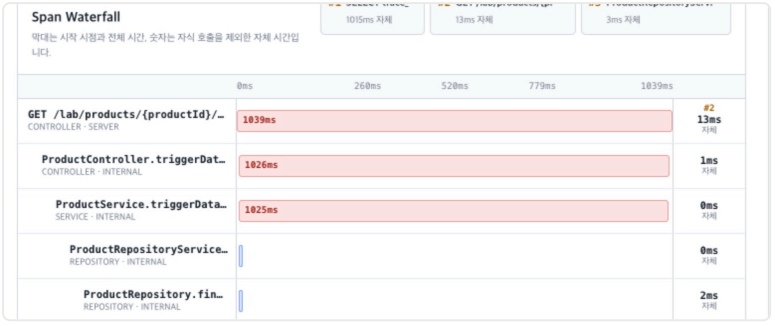
GitHub dlwldn30/stackflow ↗ | **기술** Spring Boot · React · OpenTelemetry · Docker · Playwright

예상 호출 경로 vs 실제 실행 → 정적 분석 × OTel 대조

문제 장애를 추적할 때 코드로 예상한 호출 경로가 실제 요청에서도 실행됐는지 확인할 방법이 없어, 로그에서 호출 관계와 구간별 시간을 직접 맞춰야 했습니다.

해결 예상 경로와 실제 실행을 겹쳐 보기 위해 Controller 매핑을 정적 분석하고, W3C `traceparent`를 주입해 OTel Java Agent의 span과 대조했습니다. 메타데이터는 허용 목록만 저장하고 SQL·헤더·바디는 저장하지 않으며, 직접 만든 HelpMeKorea 백엔드에 소스·빌드 파일 수정 없이 붙여 검증했습니다.

결과 예상했지만 실행되지 않은 단계까지 확인되고 장애 시나리오 6종을 스크립트로 자동 검증합니다. HelpMeKorea에서 15개 도메인·42개 엔드포인트를 경고 0건으로 탐지했고, 정적 분석으로 보이지 않던 `GET /v1/health → HealthController → Redis PING` 흐름도 트레이스로 확인했습니다.



span waterfall 화면. **1,039ms** 요청에서 어느 구간이 시간을 썼는지 자체 시간 기준으로 보여 줍니다.

오픈소스 기여 7건 머지

spring-integration #11278 · 동시 GET의 디렉터리 생성 경쟁

문제 동시 GET 요청이 같은 새 디렉터리를 만들 때 `exists()`와 `mkdirs()` 사이의 경쟁(TOCTOU)으로 정상 디렉터리에서도 실패했습니다. 이슈 #11277을 직접 등록해 수정했습니다.

판단 `Files.createDirectories()`로 교체했고, 리뷰에서 공용 기반 클래스 변경이 FTP/SFTP 예외 cause까지 바꾼다는 점을 확인했습니다. `mkdirs() || exists()`로 예외 체인을 보존하는 안도 검토한 뒤 NIO 일관성과 경쟁 방지를 택했고, 처음 검증한 file 모듈 외에 FTP/SFTP까지 확인 범위를 넓혔습니다.

결과 과한 동시성 테스트는 maintainer 의견으로 삭제했고, 기록상 file 331·FTP 92·SFTP 92 테스트와 CI가 통과해 2026-09-02에 머지됐습니다.

langchain4j #6324 · PgVector 초기화 실패 시 Connection 누수

문제 PgVector connection factory가 `getConnection()` 뒤 extension·vector 타입 초기화를 수행하는데, 반환 전에 실패하면 호출자가 Connection을 받지 못해 닫을 주체가 없었습니다.

판단 성공 시 소유권은 호출자에게 넘기고 실패 시 factory가 회수하도록 경로를 나눴습니다. 메서드 전체를 try-with-resources로 감싸면 정상 반환한 Connection까지 닫히기 때문이며, close 실패는 원래 예외에 suppressed로 붙여 최초 원인을 보존했습니다.

결과 `createStatement`·`CREATE EXTENSION`·`addVectorType` 실패 3개 테스트를 추가했고, maintainer가 정상 반환·동일 예외·suppressed close 테스트를 보강해 2026-09-10에 머지됐습니다.

spring-integration #11410 · 분산 락의 interrupt 상태 복원

문제 `JdbcLock`·`RedisLock`의 uninterruptible `lock(Duration)`이 `InterruptedException`을 무시하고 재시도하는 동안 interrupt 플래그가 지워져 호출자에게 취소 신호가 남지 않았습니다.

판단 즉시 예외를 던지면 uninterruptible 계약이 깨지고 catch 안에서 바로 `interrupt()`를 다시 걸면 다음 sleep이 즉시 실패해, 대기는 유지하고 종료 시 `finally`에서 신호를 복원했습니다.

결과 불안정한 실제 경합 테스트를 acquire 순서를 통제하는 결정적 JDBC 회귀 테스트로 바꿨고, 기록상 JDBC 42·Redis 74 테스트가 통과해 2026-09-09에 머지됐습니다.

그 밖의 머지된 PR	문제 → 영향
spring-boot #51398	AOT 파일 비교 핸들러가 스트림을 닫지 않음 → **파일 핸들 누수**
spring-kafka #4570	`DelegatingByTypeSerializer`가 `close()`를 위임 대상에 전달하지 않음 → **서드파티 Serializer 자원 미해제**
spring-ai #6797	`TextReader`가 스트림을 닫지 않음 → **문서 1건당 FD 1개 누수**
spring-integration #11353	`close()` 실패(IOException)를 무시 → **디스크가 가득 차도 정상 쓰기로 보고**, 손상 파일이 rename됨

Other Projects

Roomy — AI 기반 기숙사 룸메이트 매칭 · 생활 관리 앱

2025 · 해커톤 · **팀장 · 백엔드** · Spring Boot · WebClient

- 기숙사에서 직접 살아 본 경험으로 룸메이트 매칭과 하자 신고 아이디어를 냈고, 팀장으로 백엔드를 맡았습니다.
- 사용자·매칭·수리·공지 API와 데이터 계층을 만들고 AI 서버 연동은 WebClient로 따로 분리했습니다. 매칭은 사용자와 후보 데이터를 요청 모델로 바꿔 AI 매칭 서버에 보냅니다.
- 고장 난 시설을 사진으로 찍으면 AI가 무엇이 고장 났는지와 어떤 종류의 문제인지를 뽑아 신고 초안을 써 줍니다. 백엔드는 위치와 기존 신고를 함께 담은 분석 요청을 만들고, AI가 답한 신규 여부로 저장할지를 정합니다. AI 응답은 그대로 쓰지 않고 정책에 맞게 다듬으며, 호출이 실패하면 안전한 응답으로 넘겨 장애가 퍼지지 않게 했습니다.

Awards · Certificates · Education

Award	**2025 세종대학교 SW*AI 해커톤 은상** · 소프트웨어융합대학 소프트웨어사업단장상 출품작 「Roomy」 · 팀장 · 백엔드 담당
Program	**창조경제혁신센터 「모두의 창업」 일반기술트랙 2라운드 진출** 2026 출품작 「Cloumy」 · 기획 · 설계 · 구현
Certificate	**정보처리기사** 2026.09 · **SQLD** 제58회 2025.09 · **OPIc IH** 2026.03
Education	**세종대학교** 지능기전공학부 스마트기기전공 2022.03 ~ 2027.02 졸업예정
Activity	교내 개발 동아리 **binary** 2022.03~2024.09 · **arom** 2024.03~2026.02

이지우 · dlwldn30@naver.com · 010-2304-0895 · github.com/dlwldn30